Intro to heuristic search



Zola Mahlaza

Department of Computer Science

Feb 2026

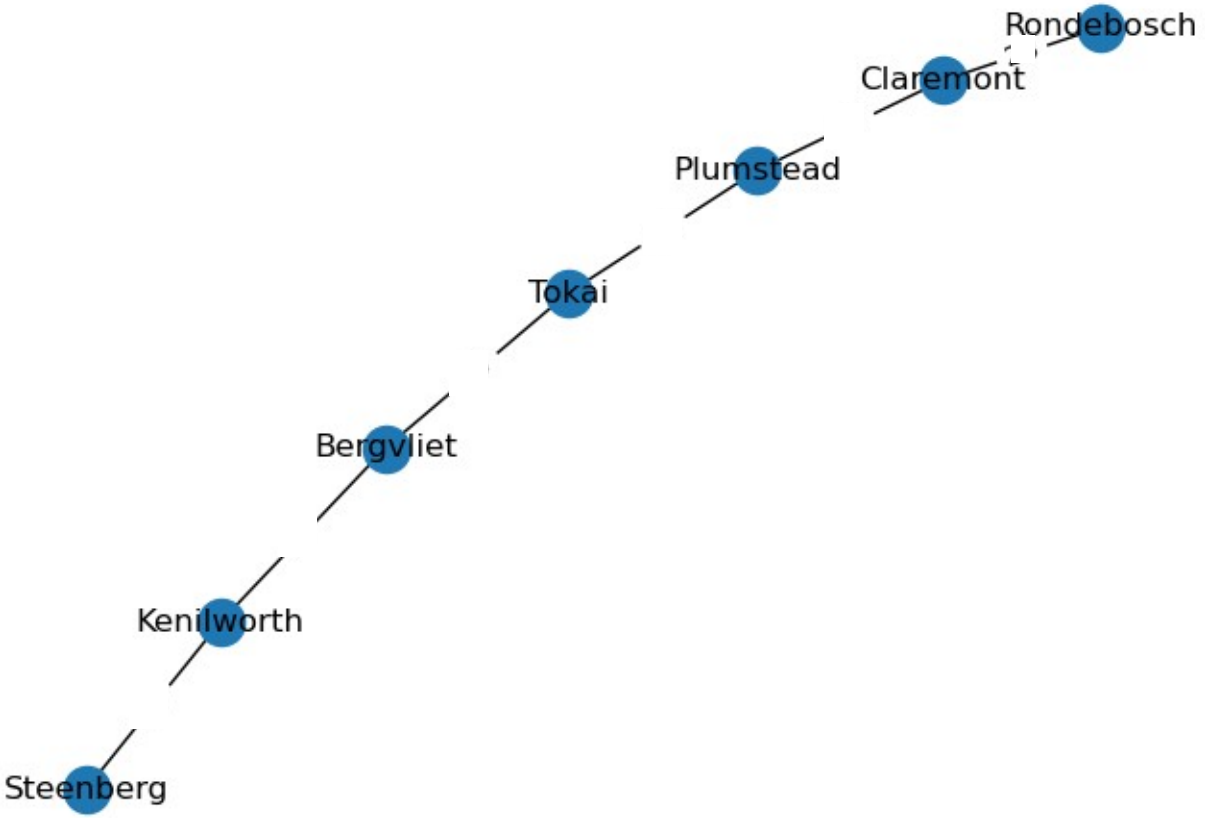
Topics

- Heuristic
- Heuristic creation
- A* search
- Beam search

Heuristic

- Search task
  - **Path**: Any sequence of actions
  - **Solution**: a path from the initial state to a goal state
  - **Optimal** solution has the lowest path cost among all solutions
- **Problem-solving task**: given a problem instance, find a(n optional) solution
- Define? the function h*(n): **true** lowest possible cost to reach a goal from node n
- No need to emphasize the balance between exploring and exploiting in search spaces.
- What would be the best action?
  - BFS guarantees finding the optimal path

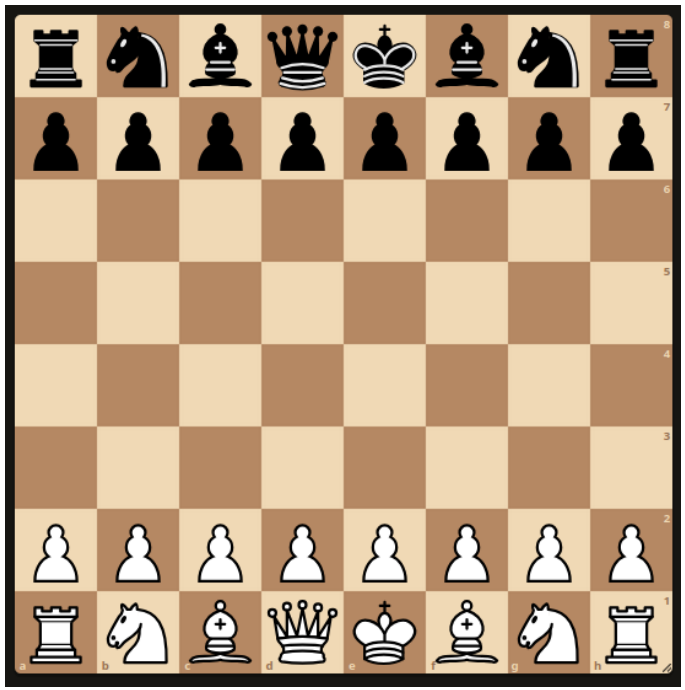
Heuristic: availability of h*(n)



$$h^*(x) = \begin{cases} 0 & x = Rondebosch \\ 1 & x = Claremont \\ 2 & x = Plumstead \\ \dots \end{cases}$$

- Initial state: Tokai
- Goal state: Rondebosch
- Greedy best-first search suffices as a solution!

- The time complexity is O(bd). b = branching factor, d = depth
- Unfortunately, you never know the exact distance to the goal!



Total (number of) winning positions

Identifying actions/states

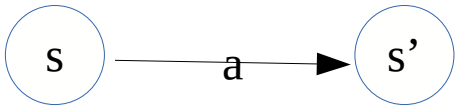
Unsolved problem!

Heuristic

- Problem: h*(n) is almost never available/accessible
- Instead, we 'estimate' it via a heuristic function, denoted h(n)
  - Define h : S → non-negative real numbers
  - Relationship between them is h(n) ≈ h*(n)
- Use domain knowledge to estimate the cost of the cheapest path from node n to a goal state
- What the limits, if any?

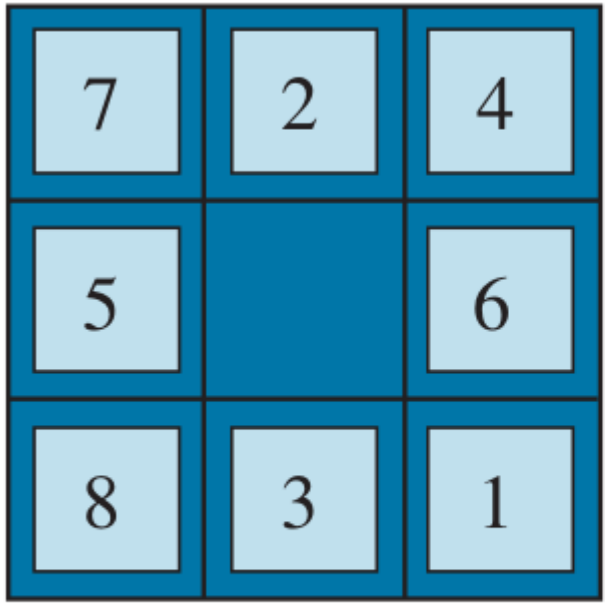
Heuristic: properties

- A heuristic needs to be **admissible** and **monotone/consistent**
- A heuristic function h(node) admissible if h(node) ≤ h*(node) for all nodes
  - It can never over estimate the true optimal cost
  - ‘Larger’ estimates risk skipping over good paths
- It is monotone if h(node) ≤ cost(node, a, node’) + h(node’) for all nodes
  - Consider the change in states:
  - Heuristic must never report a change greater that the cost of a
- Consistency is stronger that admissibility

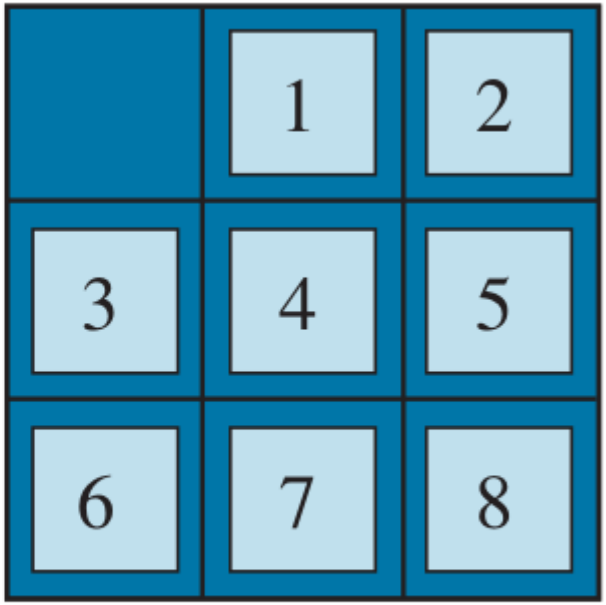


Heuristic creation: solving a problem (n-puzzle)

- Consider sliding-tile puzzle (8-puzzle)
  - tiles in a grid + one or more blank spaces, to allow movement
  - (n+1)-puzzle obtained via a n × n grid



Start State



Goal State

Heuristic creation

- State space
  - Each state is config. with numbers (1-8) and one empty space (e.g., [[1,2,3],[4,8,empty], [7,6,5]]))
  - Size of space: 9! = 362,880 states
- Actions: Left, Right, Up, Down (for the blank)
- Action cost: every action costs 1

Heuristic creation

- Initial state: Any valid state from state space suffices
- Goal State: Any config. suffices
  - Usually, you want tiles to be in numerical order, with the blank at the end
- Transition model: f :S x A -> S (e.g., moving the blank "Up" swaps it with the tile above it).

Heuristic creation

- For small n (e.g. 3), solving n-puzzles is not a challenge
  - collapse 2d grid to 1d array
  - i = # times larger number appears before a smaller number
  - If i is even in initial state, you can only reach goals where i is even (rule applies for odd i)
  - There are 9!/2 = 181400
  - Suppose each node requires 9 bytes
  - (ignoring impl. details) number of states * node size ≈ 1.x MB

Heuristic creation

- Consider a ‘small’ increase
  - 15!/2 = 10,461,394,944,000 (...trillion!)
  - Around 85.x TB
  - A heuristic becomes crucial!

Heuristic creation

- Let’s first try to create h*(n):

  A tile can move from square X to square Y if X is adjacent to Y and Y is blank.

Heuristic creation

- Let’s first try to create h*(n):

  A tile can move from square X to square Y if X is adjacent to Y and Y is blank.

If the game had “no obstacle” then this would be trivial. Manhattan distance would be h*(n)

Heuristic creation

- Common candidates for n-puzzle

  h_1 = the number of misplaced tiles

$$h_1(n) = \sum_i^n placed(tile_i)$$

Placed returns 0 if tile is in correct position, 1 otherwise

Heuristic creation

- Common candidates for n-puzzle

  h_2 = the sum of the distances of the tiles from their goal positions

$$h_2(n) = \sum_i^n manhatD(tile_i, goal_i)$$

manhatD returns the Manhattan distance

Heuristic creation

- h1 and h2 are estimates of the remaining path length
- We are not modeling all the complexities of the n-puzzle!
  - Each piece moves independently
- Number of misplaced tiles vs. distance to goal: no logic for whether two cells can occupy the same position, etc.
- **Problem relaxation**: create perfectly accurate path lengths for simplified versions of the puzzle

Heuristic creation

- Problem state of relaxed problem is larger
  - State transitions no longer tightly restricted
  - Dealing with supergraph
- Optimal problem in original problem is optimal in supergraph
  - However, reverse not true due to possible shortcuts in supergraph
- A consistent heuristic from relaxed problem is consistent in original problem

Heuristic creation

- Recall, we want to demonstrate the ff. in original problem:
  - $h(n) <= cost(n, a, n') + h(n')$
- Relaxed problem can only create shortcuts, it cannot make transitions between nodes worse
  - $cost_{relaxed}(n, a, n') <= cost(n, a, n')$ [2]
- Since h is consistent in relaxed problem
  - $h(n) <= cost_{relaxed}(n, a, n') + h(n')$
- By [2], it follows that $h(n) <= cost(n, a, n') + h(n')$

Heuristic creation

- You can relax problems in any way you prefer
  - Consider constraint: "A tile can move from square X to square Y if X is adjacent to Y and Y is blank"
- Option 1: A tile can move from square X to square Y if X is adjacent to Y. (**Manhattan distance**)
- Option 2: A tile can move from square X to square Y if Y is blank (**Teleporting to empty space**)
- Option 3: A tile can move from square X to square Y (**Teleporting to any space**)

$$h_g(n) = \begin{cases} 0 & n = goal \\ 1 + h_g(n') & otherwise \end{cases}$$

$$\sum tile_i \neq goal_i$$

$$\sum(|x_i - gx_i| + |y_i - gy_i|) \text{ for } tile_i = (x_i, y_i), goal_i = (gx_i, gy_i)$$

Heuristic creation

- You can relax problems in any way you prefer
  - Consider constraint: "A tile can move from square X to square Y if X is adjacent to Y and Y is blank"
- Option 1: A tile can move from square X to square Y if X is adjacent to Y. (**Manhattan distance**)
- Option 2: A tile can mo… X to square Y if Y is bl… **g … ce**)
- Optio… ove from s… are X to square Y (**Telepor… g to any space**)

Hamming distance

Gaschnig distance

Manhattan distance

$$\sum tile_i \neq goal_i$$

$$h_g(n) = \begin{cases} 0 & n = goal \\ 1 + h_g(n') & otherwise \end{cases}$$

$$\sum(|x_i - gx_i| + |y_i - gy_i|) \text{ for } tile_i = (x_i, y_i), goal_i = (gx_i, gy_i)$$

Heuristic creation

- Gaschnig distance cannot be treated as sum on independent states
- Moving an tile to empty position impacts whether you can also move the other tiles to the same position
  - Specificity wrt relaxed conditions
- Technically, Gaschnig is just BFS on the relaxed problem

Gaschnig distance

$$h_g(n) = \begin{cases} 0 & n = goal \\ 1 + h_g(n') & otherwise \end{cases}$$

$$h_G(n) = \sum_{i=1}^{n} cost(s_i, s_{i+1})$$

$c(s_t,s_{t+1})=1$

Reliance on the total number of steps

Heuristic creation

- There may be cases where you have multiple heuristics h_1, h_2, h_3, … that are equal good
- Choice: randomly choose 1 or use a combination
- Define h = min { h_1, h_2, h_3, … }
- If every h_i is consistent then h is consistent
- Limitation: computation overhead
  - Computing h means computing every h_i

A* search

- Most common informed search: A* algorithm
- Relies on heuristic of the form: f(n) = g(n) + h(n)
  - Idea: what is the cost until now? What do we estimate the future cost?
- Greedy best-first search kept track of g(n)
  - We used f(n) = h(n)
  - We only used g(n) to reconsider visited nodes as we go along
- Novelty: do not ignore ‘past’ cost when predicting total cost

Greedy best first search

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

A* Search

f(n) = g(n) + h(n)

```
function BEST-FIRST-SEARCH(problem, f) returns a solu                    de or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached                                       then
                reached[s] ← child
                add child to frontier
    return failure
```

Greedy best first search

start		
	obstacle	
		end

- Cost: Each move to adjacent square costs 1
- Heuristic: We use Manhattan Distance

Greedy best first search

Start - current		
	obstacle	
		end

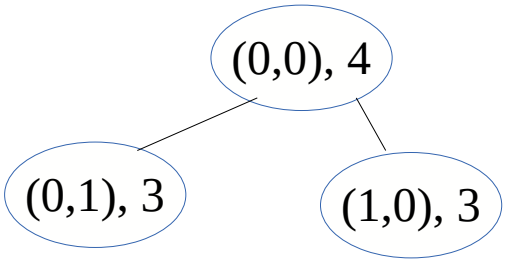
Open

(0,0)

Closed

(0,0)

Current tree – after expansion + children gen.



Greedy best first search

Start - current		
	obstacle	
		end

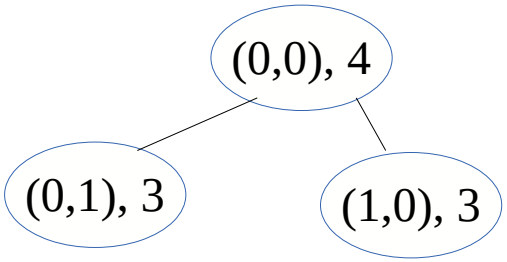
Open

(1,0)
(0,1)
(0,0)

Closed

(1,0)
(0,1)
(0,0)

Current tree – after expansion + children gen.



Greedy best first search

Start	current	
	obstacle	
		end

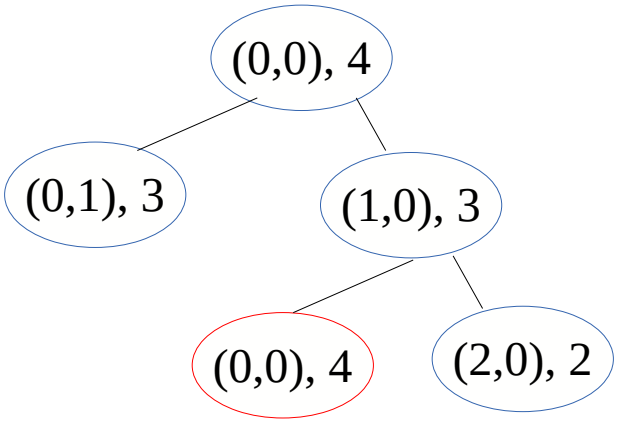
Open

~~(1,0)~~
(0,1)
(0,0)

Closed

(1,0)
(0,1)
(0,0)

Current tree – after expansion + children gen.



Visited. Only reconsider if There's shorter path

Greedy best first search

Start	current	
	obstacle	
		end

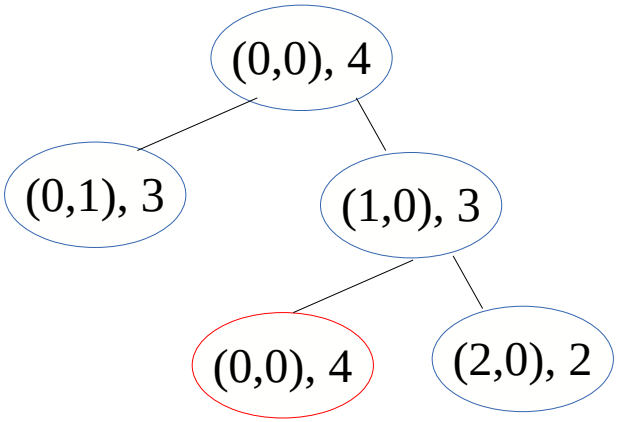
Open

(2,0)
(0,1)
(0,0)

Closed

(2, 0)
(1,0)
(0,1)
(0,0)

Current tree – after expansion + children gen.

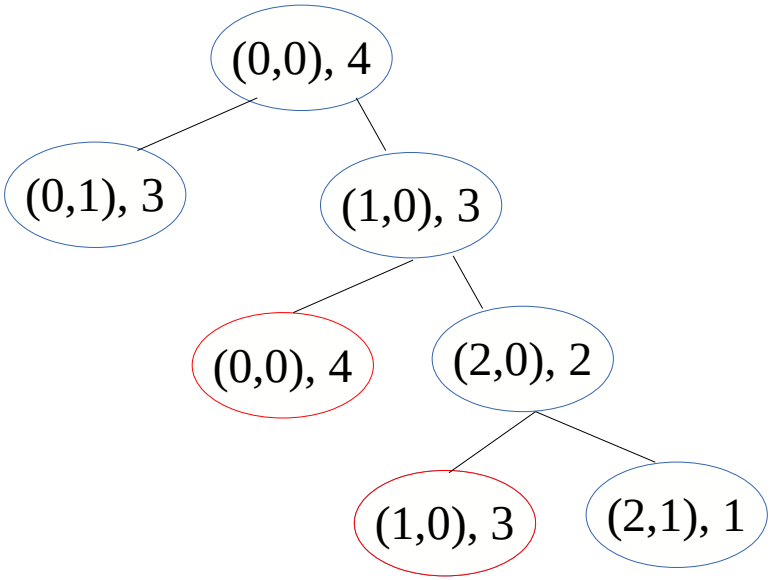


Greedy best first search

Start		current
	obstacle	
		end

Open	Closed
(2,1)	(2,1)
(0,1)	(2,0)
(0,0)	(1,0)
	(0,1)
	(0,0)

Current tree – after expansion + children gen.

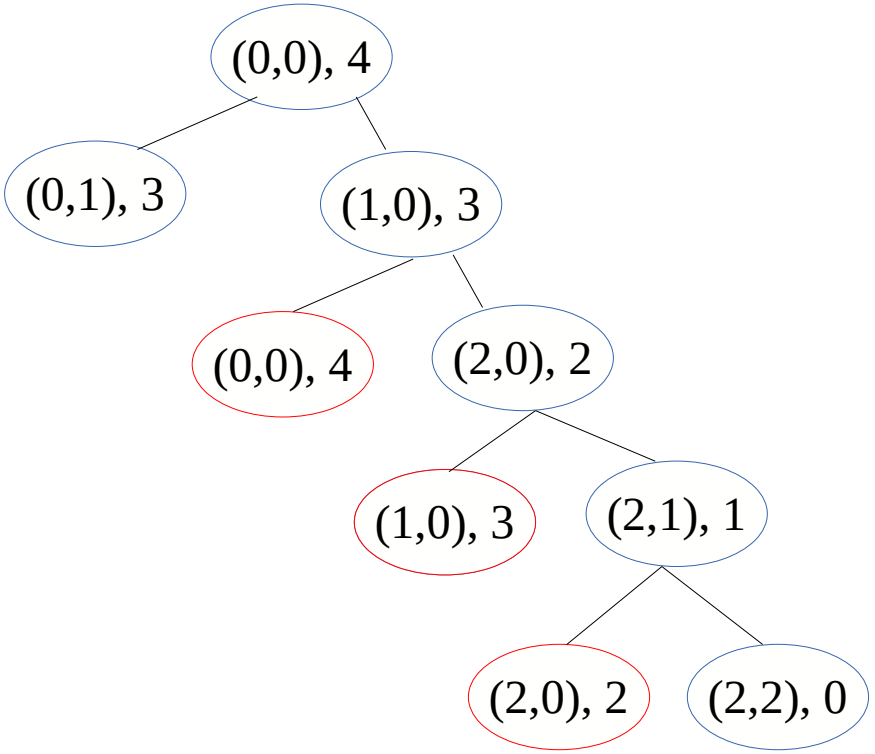


Greedy best first search

Start		current
	obstacle	
		end

Open	Closed
~~(2,1)~~	(2,1)
(0,1)	(2,0)
(0,0)	(1,0)
	(0,1)
	(0,0)

Current tree – after expansion + children gen.

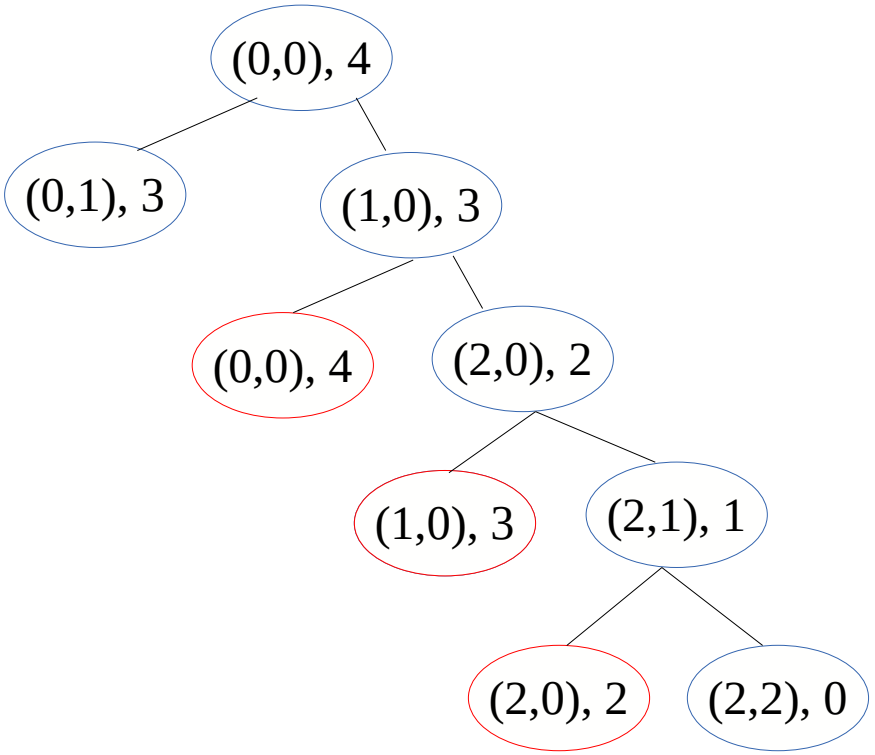


Greedy best first search

Start		
	obstacle	current
		end

Open	Closed
(2,2)	(2,1)
(0,1)	(2,0)
(0,0)	(1,0)
	(0,1)
	(0,0)

Current tree – after expansion + children gen.



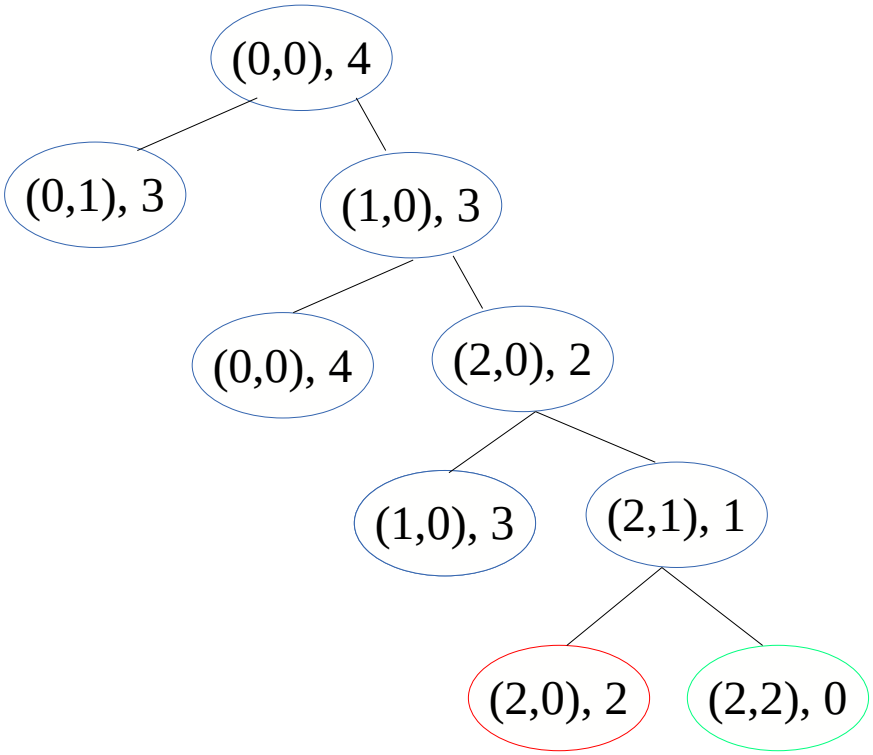
Greedy best first search

Start - current		
	obstacle	
		End - current

Open	Closed
~~(2,2)~~	(2,1)
(0,1)	(2,0)
(0,0)	(1,0)
	(0,1)
	(0,0)

Goal reached!

Current tree – after expansion + children gen.



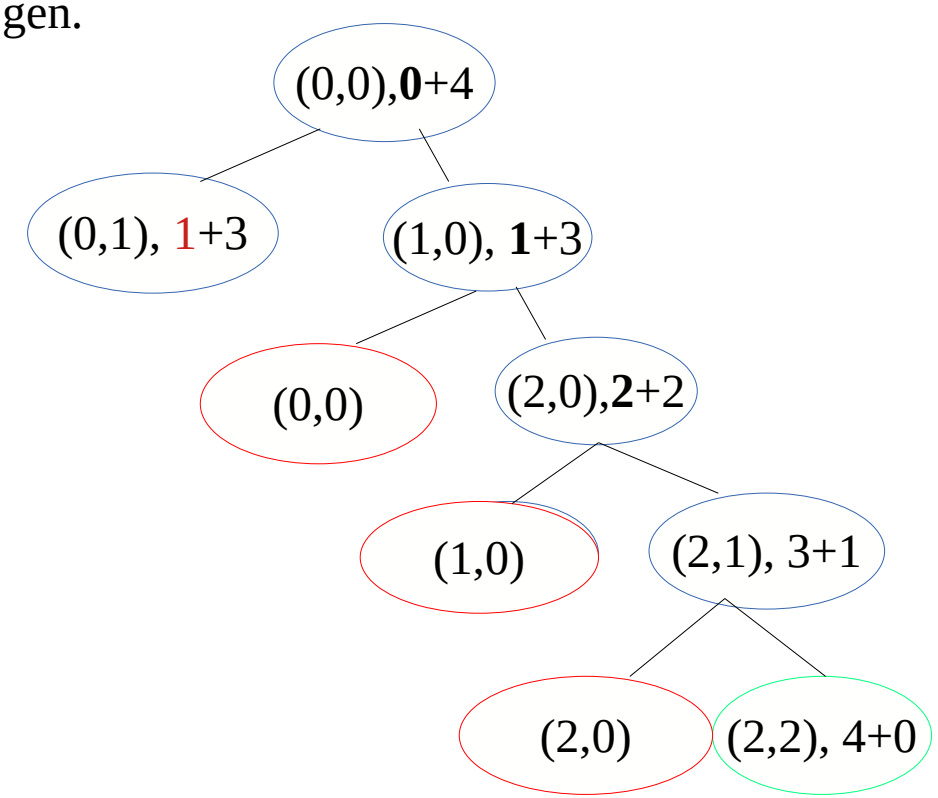
A* Search

Start - current		
	obstacle	
		End - current

Open	Closed
~~(2,2)~~	(2,1)
(0,1)	(2,0)
(0,0)	(1,0)
	(0,1)
	(0,0)

Goal reached!

Current tree – after expansion + children gen.



A* search

- Cost optimality of A* search depends on heuristic
- If heuristic is admissible and consistent then A* search is optimal

Beam search

- A* is complete and optimal but…
  - The space complexity of A* is $O(b^d)$. b = branching factor, d = depth
- If heuristic is admissible and consistent then A* search is optimal and complete (assuming finite search space)
- There are two sources of memory consumption:
  - Openlist (frontier) and closed list (reached)

Beam Search

1. sort by f
2. select only top k elements

f(n) = g(n) + h(n)

```
function BEST-F[illegible]T-SEARCH(problem, f) returns a solu[illegible]
  node ← NODE(ST[illegible]TE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached [illegible] then
        reached[s] ← child
        add child to frontier
  return failure
```

A* and beam search

- A* is complete and optimal but…
  - The space complexity of A* is $O(b^d)$. b = branching factor, d = depth
- Beam search is not complete or optimal but…
  - The space complexity is O(k.m). m = depth

Reading

- Sections 3.5 - 3.6.2

